

• SQL · LE PIÈGE DU JOIN

Tu crois que ton total est **juste** ?

Tu ajoutes un JOIN, ton SUM double. Et tu cherches le bug dans la mauvaise colonne. Ce n'est pas SQL le problème. C'est le fan-out.

● LE SYMPTÔME

Même requête. Le total passe de 100€ à 300€.

Tu calcules le chiffre d'affaires d'une commande : 100€. Tu ajoutes un LEFT JOIN sur les livraisons pour afficher le transporteur. Tu relances. SUM(montant) affiche 300€. Tu n'as touché à aucun montant.

Avant et après le JOIN

Requête	LIGNES · SUM(MONTANT)	
	Lignes	SUM(montant)
Sans le JOIN	1	100€
Avec le JOIN livraisons	3	300€

- Le montant n'a pas changé. Le **nombre de lignes** a changé. C'est tout le piège.

● LE MÉCANISME

Un côté à 1 ligne, l'autre à plusieurs : ta ligne se duplique.

La commande existe une fois. Les livraisons existent trois fois. Le JOIN ne fusionne pas : il répète la commande pour chaque livraison qui matche. Une ligne en entrée, trois en sortie.



- Le JOIN ne crée pas de faux montant. Il **recopie le vrai** autant de fois qu'il y a de lignes en face.

● LA PREUVE

Tu n'additionnes plus 1 montant. Tu additionnes le même, 3 fois.

Voilà le résultat ligne à ligne après le JOIN. La commande #C-204 apparaît sur chaque livraison, et son montant de 100€ avec elle. SUM additionne les trois. 300€. Un total qui n'existe nulle part dans la réalité.

Le résultat du JOIN, ligne par ligne

commande_id	livraison	montant	CUMUL SUM cumul SUM
#C-204	L1 · Colissimo	100€	100€
#C-204	L2 · Chronopost	100€	200€
#C-204	L3 · Relais	100€	300€

- Le vrai total, c'est 100€. Le **300€** est un artefact de la jointure, pas une vente.

À RETENIR SUM après un JOIN un-à-plusieurs compte des lignes, pas des montants.

● LE BON MOT

Ton problème a un nom : le grain de ta ligne a changé.

Le grain, c'est ce qu'une ligne représente. Avant le JOIN, 1 ligne = 1 commande. Après, sans que tu l'aies décidé, 1 ligne = 1 livraison. Tu agrèges au mauvais grain : c'est ça, le fan-out.

LE GRAIN QUE TU CROIS

1 ligne = 1 commande

Tu penses sommer des commandes. C'est ce que ta requête disait avant le JOIN.

LE GRAIN QUE TU AS

1 ligne = 1 livraison

Tu additionnes des livraisons, à l'insu de ta requête. Le JOIN a redéfini le grain pour toi.

- Quand le grain du résultat n'est plus celui que tu crois, **tout agrégat** (SUM, COUNT, AVG) ment.

● LE RÉFLEXE DE CONTRÔLE

Deux COUNT qui ne disent pas la même chose = fan-out.

Avant de faire confiance à un total, compte. COUNT(*) te donne le nombre de lignes du résultat. COUNT(DISTINCT clé) te donne le nombre de vraies entités. S'ils diffèrent, ta jointure a dupliqué.

```
SELECT
  COUNT(*)                AS lignes,      -- 3
  COUNT(DISTINCT commande_id) AS commandes -- 1
FROM commandes c
LEFT JOIN livraisons l ON l.commande_id = c.id;
```

lignes (3) ≠ commandes (1) → tu as **3 fois la même commande**.

- Même réflexe sur le total : compare ton agrégat au bon grain, ou **agrège avant** de joindre.

À RETENIR COUNT(*) différent de COUNT(DISTINCT clé), c'est le signal du fan-out. Prends l'habitude de le lancer.

● LA CORRECTION

Agrège au bon grain d'abord. Joins ensuite.

Ne joins pas une table fine à une table large puis SUM. Réduis d'abord la table large à 1 ligne par commande (une sous-requête au bon grain), et seulement après, joins. Le total reste juste.



- Règle simple : ne SUM jamais après un JOIN un-à-plusieurs. Ramène au **grain 1 pour 1** d'abord.

À RETENIR Le fan-out se corrige par l'ordre : agréger d'abord, joindre ensuite.

• LE RÉFLEXE À GARDER

Avant de SUM après un JOIN, pose-toi une question : à quel grain est ma ligne ?

C'est la seule habitude qui te protège du fan-out. Pas une règle à apprendre par cœur : un réflexe à te poser à chaque jointure avant d'agréger.

• À TOI

Ton total a déjà **explosé** après un JOIN ?

Raconte en commentaire le JOIN qui t'a piégé : sur quelle table, et comment tu l'as repéré. Je réponds à chacun.

Dis-moi ton fan-out en commentaire.

Dylan Arnaud · datafuji.com